

Eightfold Engineering Intern Assignment: Technical Design

Candidate: Ajay Balaji | Email: ajay.balaji@gmail.com | Role: Multi-Source Candidate Data Transformer

1. Pipeline Architecture

The pipeline is designed as a modular, deterministic, and explainable data flow that transforms heterogeneous, messy candidate records into structured, canonical profiles. The stages are executed in the following sequence:

- **1. Ingest & Route (Detect):** Identifies source types (CSV, ATS JSON, LinkedIn JSON, Resume PDF/DOCX/TXT) using file extension, MIME, or URL pattern heuristics.
- **2. Extract & Pre-normalize:** Extractor classes parse raw content and emit RawField evidence objects. Initial normalization (e.g., E.164 phone parsing, YYYY-MM date parsing, ISO-3166-1 country lookup, and canonical skill mapping) is performed during extraction.
- **3. Candidate Grouping (Clustering):** Groups RawFields into clusters using a Union-Find (Disjoint Set) algorithm. Records are matched if they share a normalized email, a normalized phone, or a normalized name and a social link (GitHub/LinkedIn).
- **4. Merge & Conflict Resolution:** Consolidates raw fields in each cluster into a single CanonicalProfile using a completeness-first policy and calculates an overall confidence score.
- **5. Validate (Canonical):** Mutates fixable fields and verifies the profile against the CANONICAL_JSON_SCHEMA.
- **6. Project:** Transforms the canonical profile into the desired output schema based on a runtime config.json.
- **7. Validate Output & Emit:** Verifies required fields in the projected output and handles missing fields (null/omit/error).

2. Merge & Conflict Resolution Policy

When multiple sources supply competing values for the same candidate field, the pipeline resolves conflicts deterministically:

- **Completeness-First:** Evaluates value richness using a completeness score (e.g., longer/richer strings and filled objects score higher). For example, 'Ajay Balaji' (score 0.11) wins over 'Ajay B.' (score 0.07).
- **Confidence Tiebreaker:** If completeness scores are equal, the value from the source with the higher base confidence wins (CSV: 0.90, ATS: 0.85, GitHub: 0.75, LinkedIn: 0.70, Resume: 0.65, Notes: 0.60).
- **Source Type Priority:** If a tie still exists, structured sources (CSV/ATS) are prioritized over unstructured sources.
- **Agreement Bonus:** Boosts confidence by +0.05 for each additional source that provides the same normalized value (capped at 1.0).
- **Array Fields:** Fields like emails, phones, and skills are merged via a union operation, deduplicated, and sorted by confidence descending. Skills are mapped to canonical names using an alias lookup table.

3. Runtime Custom-Output Config (Projection Layer)

The projection layer reshapes the internal CanonicalProfile into the final output without changing any engine code. The runtime config.json supports:

- **Field Selection & Renaming:** Projects a subset of fields and maps them using the 'from' key (e.g., mapping emails[0] to primary_email or skills[].name to a flat string list).
- **Per-Field Renormalization:** Applies additional normalization during projection (e.g., E164 for phones, canonical for skills, or iso3166 for countries).
- **Metadata Toggles:** Top-level include_confidence and include_provenance booleans toggle the inclusion of confidence scores and the detailed provenance array.
- **Missing Value Handling:** The on_missing key dictates behavior when a required field is null or missing: 'null' (inject null), 'omit' (exclude field), or 'error' (raise a blocking ValueError).

4. Edge Cases Handled

- **Garbage/Malformed Sources:** Catching file read and parsing exceptions. Malformed rows/objects degrade gracefully by skipping bad records rather than crashing the pipeline.
- **Date & Year Validation:** Ensures experience start dates do not exceed end dates (nulling the end date and raising a warning if they do). Validates that education graduation years are numeric.
- **No Reliable Identifiers:** If a candidate lacks emails, phones, and social links, the pipeline falls back to generating a random UUID4 for the candidate_id, flagging it in the provenance with a low confidence score (0.1).
- **Scale & Batching:** The Union-Find grouping algorithm allows thousands of candidate records across massive files to be clustered and processed in parallel, maintaining high throughput and memory efficiency.